

Nome: Laura Campos Pontara Lopes

Resenha: Design by Contract (DbC)

Esta resenha analisa o capítulo "Design by Contract", de autoria de Bertrand Meyer, que apresenta uma metodologia rigorosa para aumentar a confiabilidade de sistemas de software através de uma metáfora contratual. A tese central de Meyer é que a construção de software deve ser entendida como uma sucessão de decisões documentadas entre componentes, onde cada interação é regida por obrigações e benefícios mútuos. O autor argumenta que a falta de rigor na definição dessas interfaces é a causa primária da fragilidade em sistemas complexos e que as técnicas formais de especificação, muitas vezes ignoradas na prática comum, podem ser integradas de forma elegante ao ciclo de desenvolvimento através do mecanismo de asserções.

O fundamento do Design by Contract (DbC) reside em três pilares: pré-condições, pós-condições e invariantes de classe. As pré-condições definem os requisitos que o cliente deve satisfazer para que uma chamada de rotina seja considerada válida. Por outro lado, as pós-condições descrevem as propriedades que o fornecedor garante entregar após a execução, assumindo que a pré-condição foi respeitada. Complementarmente, os invariantes de classe representam restrições de integridade que devem ser preservadas por todas as instâncias da classe em estados estáveis. Meyer destaca que essa estrutura protege ambos os lados: o cliente tem a garantia de um resultado, enquanto o fornecedor limita sua responsabilidade apenas aos casos previstos no contrato.

Uma das contribuições mais provocativas do texto é a crítica contundente à chamada "programação defensiva". Meyer sustenta que o hábito de incluir verificações redundantes em todos os módulos aumenta a complexidade do código e, paradoxalmente, reduz a confiabilidade ao ocultar a lógica de negócio em um "oceano de verificações de erro". Ao aplicar o DbC, a responsabilidade é atribuída explicitamente a um único elemento. Se uma pré-condição é violada, o erro é categoricamente do cliente; se a pós-condição falha, a culpa é do fornecedor. Essa clareza permite a construção de arquiteturas mais limpas e modulares, onde os componentes podem ser reutilizados com total confiança em sua especificação de interface.

O autor também estende a teoria do contrato para o contexto da herança e do polimorfismo, introduzindo o conceito de "subcontratação honesta". Na visão de Meyer, uma subclasse que redefine um método está agindo como um subcontratado. Para manter a

integridade do sistema, o subcontratado deve respeitar a regra da redefinição: as pré-condições só podem ser enfraquecidas (exigir menos do cliente) e as pós-condições só podem ser fortalecidas (entregar mais ou melhor resultado). Essa restrição formal é o que impede que o dynamic binding se torne uma fonte de erros imprevisíveis, garantindo que qualquer objeto de uma classe derivada se comporte de maneira consistente com a expectativa gerada pela classe base.

No que tange ao tratamento de exceções, Meyer propõe uma abordagem disciplinada baseada na incapacidade de cumprir um contrato. Ele rejeita o uso de exceções como meras estruturas de controle de fluxo, como ocorre em linguagens como Ada ou CLU, criticando-as por permitirem que rotinas falhem silenciosamente ou retornem estados inconsistentes. No modelo de Eiffel apresentado, existem apenas duas reações legítimas a uma exceção: a retomada (resumption), onde a rotina tenta uma nova estratégia para cumprir o contrato original, ou o "pânico organizado", onde a rotina restaura o invariante da classe para garantir um estado estável e reporta a falha ao chamador.

Em suma, a obra de Meyer é um manifesto pela simplicidade e pelo rigor matemático na engenharia de software. Ao aceitar que funções podem ser parciais e ao exigir que essas limitações sejam explicitadas em contratos claros, o autor oferece um caminho para sistemas que não apenas funcionam, mas que são compreensíveis, documentáveis e robustos por design. O legado do Design by Contract transcende a linguagem Eiffel, influenciando profundamente a forma como pensamos sobre interfaces, testes e arquitetura de software na atualidade.

Referências Bibliográficas

MEYER, Bertrand. **Design by Contract**. Interactive Software Engineering Inc., [S. l.: s. n.], [1997]. 50 p. Disponível em: conteúdo fornecido pelo usuário. Acesso em: 10 maio 2026.